

RIPASSO TECNICO

Angular v22 (2026) per uno sviluppatore Vue

Manuale di riferimento — concetti, confronti con Vue, tour del progetto
"Task Board" e basi di testing con Vitest

Costruito a partire da una conversazione con Claude
Progetto di riferimento: `angular-vue-crash-project`
Agosto 2026

Indice

1. Contesto e obiettivo del ripasso
2. Angular v22 (2026): panoramica dello stato dell'arte
3. Angular vs Vue: cheat sheet dei concetti
4. La dependency injection e il prop drilling
5. Observable/RxJS: cosa sono e che rapporto hanno coi Signal
6. Tour del codice — progetto "Task Board"
 - a. TaskStore — stato condiviso e reattività dei Signal
 - b. TaskItem — input() e injection context
 - c. TaskList — httpResource() ed effect()
 - d. TaskForm — Signal Forms
 - e. AuthService / auth-guard — guard funzionali
 - f. TaskDetail — route param binding
 - g. App, main.ts, Login — bootstrap dell'applicazione
7. Testing — basi di TDD, Vitest e TestBed
 - a. Concetti fondamentali
 - b. task-store.spec.ts — unit test puro
 - c. auth-guard.spec.ts — testare una funzione con inject()
 - d. task-form.spec.ts — component test e simulazione DOM
 - e. app.spec.ts — smoke test
 - f. task-list.spec.ts — il deadlock che abbiamo risolto insieme
8. Concetti generali aggiuntivi
 - a. Incapsulamento dei componenti (View Encapsulation)
 - b. Tailwind CSS installato per davvero: cosa succede in pratica
 - c. Design token: cosa sono e come li usiamo
 - d. Shadow DOM: come funziona, confronto con Vue, relazione con i design token
 - e. L'esperimento ShadowDom, capitolo 2: stavolta funziona
9. Appendice — note operative (setup ambiente)

1. Contesto e obiettivo del ripasso

Sei un UI/UX Designer e Frontend Developer freelance, esperto di **Vue** (reattività, composables, Pinia). Non usi Angular da circa dieci anni, e ti è stato proposto un progetto di lavoro in Angular: questo

manuale nasce da un ripasso mirato, prima di iniziare. Al tempo in cui usavi Angular, **RxJS** era il tuo punto debole — è uno dei fili conduttori di questo documento.

Non hai ancora informazioni sullo stack specifico del progetto di lavoro proposto (NgRx? Nx/monorepo? Angular Material? SSR?) — questo manuale copre i fondamentali moderni, da integrare con dettagli mirati quando lo stack reale sarà noto.

2. Angular v22 (2026): panoramica dello stato dell'arte

- **Signals** (`signal()` , `computed()` , `effect()`) stabili dalla v20: oggi il modo raccomandato di gestire stato reattivo locale. Concettualmente vicini a `ref()` / `computed()` / `watch()` di Vue.
- **Zoneless** stabile dalla v20.2: niente più `zone.js`, `OnPush` è il default.
- **Componenti standalone** sono la norma dalla v17: niente più `NgModule` obbligatori.
- **Nuovo control flow nei template**: `@if` / `@for` / `@switch` / `@let` / `@defer` al posto di `*ngIf` / `*ngFor` / `*ngSwitch` , stabile dalla v18.
- **Signal Forms** (novità 2026): form basati su signal, validatori dichiarativi (`required` , `minLength` , `maxLength` , `min` , `max` , `pattern` , `email`), binding con la direttiva `[formField]` .
- **resource()** / **httpResource()** / **rxResource()** : data fetching dichiarativo via signal (`isLoading()` , `error()` , `value()` , `hasValue()`).
- **Build**: Vite/esbuild di default dalla v17.
- **Test**: Vitest sta sostituendo Karma/Jasmine come runner raccomandato.
- **DI**: `inject()` è oggi il modo più comune di ottenere le dipendenze, anche fuori dal costruttore.
- **Routing**: i guard oggi si scrivono come funzioni (`CanActivateFn`) con `inject()` dentro, non più come classi.

RXJS: LA BUONA NOTIZIA

RxJS non è più il collante universale che era 10 anni fa. Regola pratica 2026: **Signals per lo stato locale/UI** (component state, form, binding reattivi semplici), **RxJS solo per il "mondo esterno"** (composizione di chiamate HTTP complesse, retry/cancellazione, WebSocket/stream realtime, coordinamento di più sorgenti asincrone). Per la maggior parte dei casi CRUD oggi non serve più padroneggiare RxJS in profondità.

3. Angular vs Vue: cheat sheet dei concetti

Vue	Angular
<code>ref()</code>	<code>signal()</code>
<code>computed()</code>	<code>computed()</code>
<code>watch / watchEffect</code>	<code>effect()</code>
<code>defineProps()</code>	<code>input()</code> (per-campo, non un "props bag" unico)
<code>defineEmits()</code>	<code>output()</code>
<code>defineModel()</code>	<code>model()</code>
<code>v-if</code>	<code>@if</code>
<code>v-for</code> (con <code>:key</code> opzionale)	<code>@for</code> (<code>track</code> obbligatorio)
nessun equivalente diretto	<code>@empty</code> nel blocco <code>@for</code>
<code>composable / provide / inject</code>	<code>inject()</code> (DI gerarchica nativa)
<code>composable custom / TanStack Query</code>	<code>resource()</code> / <code>httpResource()</code>
<code>VeeValidate</code> + <code>v-model</code>	Signal Forms + <code>[formField]</code>
<code>beforeEnter</code> (Vue Router)	<code>CanActivateFn</code> (guard funzionale)
<code>reactive()</code> (deep, via Proxy)	nessun equivalente — vedi §6a

Confronto generale: oggi le due reattività sono concettualmente simili (entrambe puntano su primitive fine-grained). Vue resta più leggero/flessibile da imparare; Angular è più opinionato (TypeScript quasi obbligatorio, DI integrata, routing/HTTP inclusi) — pensato per team grandi dove serve uniformità. Bundle size: Vue ~35KB gzip, Angular ~62KB gzip.

4. La dependency injection e il prop drilling

Angular ha la **dependency injection gerarchica** nativa fin dalla v1 — l'equivalente (più potente) di `provide()` / `inject()` in Vue, integrato nel sistema dei componenti:

- Crei una classe/service qualsiasi, la registri (`providers: [...]` sul componente "radice" del sottoalbero, oppure `providedIn: 'root'` per globale).
- Qualsiasi componente discendente, a qualunque profondità, la inietta direttamente (`inject(MioService)`) senza passare da `@Input()` / `@Output()` nei componenti intermedi.
- Pratica moderna: mettere `signal()` dentro il service → uno "store reattivo" scoped, l'equivalente di un mini-Pinia locale.
- Per stato globale strutturato su tutta l'app: **NgRx SignalStore**, evoluto sui signals invece che su RxJS/reducer classici — stessa filosofia di un intero store Pinia.

5. Observable/RxJS: cosa sono e che rapporto hanno coi Signal

Un **Observable** è il tipo primitivo di RxJS: rappresenta uno **stream di valori nel tempo** (zero, uno, o infiniti valori) — a differenza di una Promise, che risolve esattamente una volta.

- **Lazy**: non fa nulla finché qualcuno non chiama `.subscribe()`. Una Promise parte a eseguire non appena creata (eager).
- **Cancellabile nativamente**: `.subscribe()` restituisce una subscription disiscrivibile (`.unsubscribe()`).

Nel vecchio stile Angular, `HttpClient.get(url)` restituisce un `Observable<T>`: dovevi fare `.subscribe()` a mano, gestire loading/errori con variabili proprie, disiscriverti in `ngOnDestroy`.

RAPPORTO CON HTTPRESOURCE()

`httpResource()` non sostituisce gli Observable, li nasconde. Sotto il cofano `HttpClient` è ancora basato su Observable — la `subscribe()` la fa Angular per te. La famiglia completa:

- `httpResource()` — caso semplice, dichiarativo, signal in/signal out
- `rxResource()` — quando serve un vero Observable con operatori RxJS (`debounceTime`, `switchMap` per un tipeahead), risultato comunque esposto come signal
- `resource()` — versione generica, per qualsiasi loader asincrono
- `toSignal()` / `toObservable()` — ponti espliciti tra i due mondi (es. `router.events`, `form.valueChanges`)

Regola pratica: signal per stato locale/UI semplice, RxJS quando devi *comporre* sorgenti asincrone (tipeahead con cancellazione, WebSocket, coordinamento di più fonti).

6a. TaskStore — stato condiviso e reattività dei Signal

src/app/core/task-store.ts

`TaskStore` è un `Injectable` qualsiasi con dei signal dentro. Grazie a `providedIn: 'root'`, Angular crea un'unica istanza condivisa per tutta l'applicazione — esattamente come un singolo store Pinia.

```
@Injectable({ providedIn: 'root' })
export class TaskStore {
  private readonly _tasks = signal<Task[]>([]);
  readonly tasks = this._tasks.asReadonly();
  readonly completedCount = computed(
    () => this._tasks().filter(t => t.completed).length
  );

  add(title: string): void {
    this._tasks.update(current => [{ id: nextId, title, completed: false },
    ...current]);
  }
  // toggle, remove, hydrate: stesso pattern immutabile
}
```

Come si legge un signal

Per leggere un signal lo chiami come una funzione, sempre — sia nel `.ts` che nel template (`tasks()` , non `tasks.value` come in Vue script, e non l'unwrap automatico dei template Vue). Niente magia del compilatore: è esplicito ovunque.

`asReadonly()` : non è lo stesso riferimento JS

`tasks` e `_tasks` sono **due funzioni distinte** (`tasks !== _tasks`). `asReadonly()` crea un nuovo accessor che punta alla **stessa cella di stato interna** di `_tasks` : leggono sempre lo stesso valore corrente perché guardano lo stesso posto in ogni istante, non perché sono la stessa variabile — più vicino a due closure che catturano la stessa variabile esterna, che a `let a = b` .

ATTENZIONE — NIENTE REATTIVITÀ "DEEP" COME IN VUE

I signal **non** hanno un equivalente di `reactive()`. Non c'è un Proxy che intercetta mutazioni a proprietà annidate: Angular decide se notificare i consumer confrontando (`Object.is`) il vecchio e il nuovo riferimento di livello superiore.

```
const state = signal({ user: { settings: { theme: 'dark' } } });

// NON notifica nessun consumer — mutazione silenziosa, invisibile:
state().user.settings.theme = 'light';

// SÌ, perché sostituisce il riferimento in cima:
state.set({ user: { settings: { theme: 'light' } } });
```

Per questo `add / toggle / remove` in `TaskStore` ricostruiscono sempre un nuovo array (`.filter`, `.map`, `spread`) invece di mutare quello esistente — non è stile, è l'unico modo per far scattare la reattività. Con un oggetto annidato vale a cascata: bisogna ricostruire un nuovo riferimento a ogni livello fino alla radice (structural sharing, come Redux o Immer). `NgRx SignalStore` gestisce questo internamente con `patchState()`. Esiste un secondo parametro opzionale `signal(v, { equal: fn })` per personalizzare il confronto, ma non aggiunge tracking profondo — cambia solo il criterio "sono diversi?", sempre sul valore intero.

6b. TaskItem — input() e injection context

```
src/app/features/task-item/task-item.ts
```

Dimostra entrambe le tecniche di comunicazione: il dato `task` arriva dal padre come `input()` (come una prop Vue), ma le azioni `toggle()` / `remove()` non passano da un `output()` che risale al padre — `TaskItem` inietta `TaskStore` direttamente. Se fosse annidato 5 livelli più in profondità, non cambierebbe nulla: nessun componente intermedio dovrebbe conoscere l'evento.

INPUT() NON È "PROPS"

Con `defineProps<{ task: Task }>()` in Vue, `props` è un oggetto reattivo intero — leggi `props.task` direttamente, senza `.value`. Angular non ha un "props bag" unico: ogni `input()` è il suo signal indipendente. L'equivalente più preciso non è `props.task`, ma un `toRef(props, 'task').value` — l'estrazione di un singolo campo reattivo.

INJECTION CONTEXT: DOVE PUOI CHIAMARE INJECT()

`inject()` funziona solo in un injection context: (1) come iniziatore di campo, (2) dentro il costruttore, oppure (3) dentro una funzione factory speciale — es. un guard funzionale invocato dal router, che crea lui stesso il contesto. Non è chiamabile dentro un metodo qualsiasi come `toggle()`. In Vue `useTaskStore()` puoi chiamarlo più liberamente nel `setup()`.

Mappatura binding nel template

Angular	Vue
<code>[class.completed]="task().completed"</code>	<code>:class="{ completed: task.completed }"</code>
<code>[checked]="task().completed"</code>	<code>:checked="task.completed"</code>
<code>(change)="toggle()"</code>	<code>@change="toggle"</code>
<code>[routerLink]="['/tasks', task().id]"</code>	<code>:to="/tasks/\${task.id}"</code>

6c. TaskList — `httpResource()` ed `effect()`

`src/app/features/task-list/task-list.ts`

```
protected readonly remoteTasks = httpResource<RemoteTodo[]>(
  () => 'https://jsonplaceholder.typicode.com/todos?_limit=5',
);

constructor() {
  effect(() => {
    const data = this.remoteTasks.value();
    if (data && this.store.totalCount() === 0) {
      this.store.hydrate(data.map(t => ({ id: t.id, title: t.title, completed:
t.completed })));
    }
  });
}
```

La funzione factory di `httpResource` viene rieseguita ogni volta che un signal letto al suo interno cambia — qui non ne legge nessuno, quindi la richiesta parte una volta sola. Se leggesse un signal `page()`, rifarebbe automaticamente la fetch a ogni cambio pagina (reattività alle dipendenze, come una `queryKey` di TanStack Query).

`effect()` è il tuo `watchEffect()`: gira subito, poi ogni volta che un signal letto al suo interno cambia, con dipendenze tracciate automaticamente. Deve stare in un injection context (qui il costruttore). Nota: legge **due** signal (`remoteTasks.value()` e `store.totalCount()`) — quindi si riattiva anche dopo ogni `add / toggle / remove`, non solo alla risposta HTTP; la guardia `totalCount() === 0` rende l'idratazione "una sola volta".

Il template — tre dettagli

- `@if / @else if` senza `@else` finale: la lista sotto (`@for`) non è dentro la catena condizionale — è indipendente, perché lo store può già avere dati locali.
- `@for (task of store.tasks(); track task.id)` — `track` è obbligatorio (a differenza del `:key` facoltativo di Vue).
- `@empty { ... }` — Vue non ha un equivalente nativo: renderizza automaticamente quando l'iterable è vuoto.

6d. TaskForm — Signal Forms

src/app/features/task-form/task-form.ts

```
protected readonly model = signal<NewTaskModel>({ title: '' });

protected readonly taskForm = form(this.model, (schemaPath) => {
  required(schemaPath.title, { message: 'Il titolo è obbligatorio.' });
  minLength(schemaPath.title, 3, { message: 'Il titolo deve avere almeno 3 caratteri.' });
});
```

`schemaPath` rispecchia la forma del modello (`schemaPath.title` ↔ `model().title`) ma **non contiene i valori**: è un percorso usato solo per attaccare le regole al campo giusto.

COLLEGAMENTO CON LA REATTIVITÀ "DEEP" (§6A)

Signal Forms risolve il problema della reattività annidata **non** con un Proxy magico, ma generando esplicitamente **un signal indipendente per ogni campo**, in un albero che rispecchia la forma del modello: `taskForm()` (form intero) dà lo stato a livello di form (`.invalid()`); `taskForm.title()` (singolo campo) dà stato separato (`.touched()` , `.invalid()` , `.errors()`). È una struttura costruita apposta per aggirare il limite dei signal di base, non reattività deep "gratuita".

VALIDAZIONE: COSA VIENE "GRATIS"

Un set di validatori comuni pronti all'uso (`required` , `minLength` , `maxLength` , `min` , `max` , `pattern` , `email`) — per i casi standard non serve nessuna libreria esterna. Non è una novità 2026 in sé (i Reactive Forms classici li avevano già); la novità è lo stile dichiarativo a schema/signal. Per regole custom (cross-field, asincrone, business-specific) scrivi tu la funzione, ma l'infrastruttura (`touched/dirty`, quando rivalidare, come esporre gli errori) resta a carico del framework — a differenza di `VeeValidate`, comunque una dipendenza esterna anche per i casi semplici.

6e. AuthService / auth-guard — guard funzionali

`src/app/core/auth.service.ts` e `src/app/core/auth-guard.ts`

`AuthService` è lo stesso pattern di `TaskStore` in miniatura (signal privato + `asReadonly()`).

```
export const canActivateNewTask: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) return true;

  return router.createUrlTree(['/login']); // redirect tipizzato, non solo `false`
};
```

Un tempo un guard era una **classe** che implementava `CanActivate`, registrata nei `providers`. Oggi è una **funzione** con la forma `CanActivateFn`, referenziata direttamente in `canActivate: [...]` sulla rotta. `inject()` qui è legittimo anche fuori da costruttore/campo: il router invoca le guard dentro un injection context che crea lui stesso — terzo caso valido, oltre ai due già visti in §6b.

`router.createUrlTree(['/login'])` invece di un semplice `false`: modo idiomatico per "reindirizzare" da un guard, che il router sa interpretare automaticamente.

6f. TaskDetail — route param binding

src/app/features/task-detail/task-detail.ts

```
readonly id = input.required<string>(); // sempre stringa: gli URL non hanno tipi
protected readonly task = computed(() => this.store.findById(Number(this.id())));
```

WITHCOMPONENTINPUTBINDING() — OPT-IN, NON DEFAULT

Va abilitato esplicitamente in `app.config.ts` (`provideRouter(routes, withComponentInputBinding())`), altrimenti `TaskDetail` non riceverebbe mai `id` come `input()` e servirebbe tornare a `inject(ActivatedRoute) + route.snapshot.paramMap.get('id')`. Equivalente Vue Router: `{ path: '/tasks/:id', props: true }` — anche lì esplicito.

TRACKING DELLE DIPENDENZE ATTRAVERSO CONFINI DI FUNZIONE

Il `computed()` sopra dipende da **due** signal, ma solo uno (`id`) è letto direttamente nel suo corpo. `store.findById(...)` è un metodo qualsiasi che al suo interno legge `this._tasks()` — quella lettura "attraversa" la chiamata di funzione e viene comunque registrata come dipendenza del `computed` esterno, purché la catena di chiamate sia sincrona. Il tracking automatico osserva quali signal vengono letti durante l'esecuzione, non solo quelli scritti lessicalmente dentro la funzione — stesso principio dell' `effect()` di §6c, solo più nascosto.

Nel template, `@if (task(); as t)` salva il risultato (già "narrowed" per escludere `undefined`) in una variabile locale al blocco — Vue non ha un vero equivalente diretto di questa sintassi.

6g. App, main.ts, Login — bootstrap dell'applicazione

```
// main.ts — l'equivalente di createApp(App).mount('#app')
bootstrapApplication(App, appConfig).catch(err => console.error(err));

// app.config.ts — l'equivalente della catena app.use(router).use(pinia)...
export const appConfig: ApplicationConfig = {
  providers: [
    provideZonelessChangeDetection(),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(),
  ],
};
```

Non ogni campo di un componente deve essere un signal: `title = 'Task Board'` in `App` è una stringa semplice — non cambia mai dopo la creazione, quindi non serve reattività. Il signal serve solo per stato che cambia nel tempo e a cui qualcosa deve reagire.

PROTECTED, NON PRIVATE — PERCHÉ CONTA

In `Login`, `protected readonly authService = inject(AuthService)` è `protected` perché il template legge `authService.isLoggedIn()` direttamente sulla classe. Con `strictTemplates: true` (in `tsconfig.json`), Angular verifica l'accessibilità a compile-time — se fosse `private`, la compilazione del template fallirebbe. A differenza di `<script setup>` in Vue (dove tutto è automaticamente visibile al template), qui serve esporre esplicitamente con `protected` o `public`.

Il giro completo di autenticazione: click su "Nuovo task" → `canActivateNewTask` legge `authService.isLoggedIn()` → `false` → redirect via `UrlTree` a `/login` → click su "Login" → `authService.login()` → il signal diventa `true` → `router.navigateByUrl('/new')` → il guard rigira, questa volta passa.

7a. Testing — concetti fondamentali

Un test automatico è codice che esegue altro codice e verifica il risultato, al posto di farlo a mano nel browser ogni volta.

TDD — TEST-DRIVEN DEVELOPMENT

Ciclo **Red** → **Green** → **Refactor**: (1) scrivi il test *prima* del codice, fallisce (rosso); (2) scrivi il minimo indispensabile per farlo passare (verde); (3) migliori il codice tenendo i test verdi come rete di sicurezza. *Nota onesta*: il progetto Task Board non è stato scritto con vero TDD (test e codice generati insieme) — ma gli strumenti sono gli stessi che useresti facendo TDD sul serio.

Terminologia

- **Unit test**: testa un singolo pezzo isolato (una funzione, una classe) senza il resto dell'app.
- **Component test**: renderizza un componente vero (DOM incluso) e verifica il template.
- **Smoke test**: il test più minimale possibile — verifica solo "si accende senza esplodere".
- **Test isolation**: ogni test deve poter girare da solo, in qualunque ordine — da qui i `beforeEach` che ricreano stato pulito prima di ogni `it()`.
- **Pattern AAA**: Arrange (prepari lo stato) → Act (esegui l'azione) → Assert (verifichi il risultato).

API Vitest (analogo più veloce di Jest, nativo su Vite)

Funzione	Scopo
<code>describe(nome, fn)</code>	raggruppa test correlati (organizzativo)
<code>it(nome, fn)</code> (alias <code>test</code>)	un singolo caso di test
<code>expect(v).matcher(atteso)</code>	l'asserzione: <code>toBe</code> , <code>toEqual</code> , <code>toContain</code> , <code>toBeInstanceOf</code> , <code>toBeTruthy</code> , <code>toHaveBeenCalledWith</code>
<code>beforeEach</code> / <code>afterEach</code>	setup/teardown prima o dopo ogni <code>it()</code>
<code>vi.spyOn(obj, 'metodo')</code>	osserva le chiamate a un metodo reale senza sostituirne il comportamento di default

TestBed (Angular)

API	Scopo
<code>TestBed.configureTestingModule({imports, providers})</code>	dichiara cosa serve per il test
<code>TestBed.inject(Classe)</code>	ottiene un'istanza via DI

<code>TestBed.createComponent(Comp)</code>	crea un'istanza renderizzata, restituisce una <code>fixture</code>
<code>fixture.detectChanges()</code>	forza un ciclo di change detection
<code>fixture.whenStable()</code>	aspetta che l'app diventi stabile
<code>fixture.nativeElement</code>	l'elemento DOM reale, <code>querySelector</code> -abile
<code>TestBed.runInInjectionContext(fn)</code>	esegue <code>fn</code> dentro un injection context artificiale

7b. task-store.spec.ts — unit test puro

Testare un `Injectable` semplice non richiede quasi nulla di speciale: `TestBed.inject` basta, non serve un componente.

```
beforeEach(() => {
  TestBed.configureTestingModule({});
  store = TestBed.inject(TaskStore);
});

it('toggle() inverte lo stato completed del task giusto', () => {
  store.add('Task A');
  store.add('Task B');
  const [taskB, taskA] = store.tasks(); // add() mette in cima, quindi B è primo
  store.toggle(taskA.id);
  expect(store.findById(taskA.id)?.completed).toBe(true);
});
```

ATTENZIONE ALL'ORDINE NELLA DESTRUTTURAZIONE

`const [taskB, taskA] = store.tasks()` riflette l'ordine reale dell'array, non l'ordine di inserimento — `add()` mette in cima, quindi l'ultimo aggiunto è il primo elemento. Invertire i nomi per errore farebbe passare il test "per caso" solo se il bug cercato non tocca quella asserzione: un esempio concreto di test che sembra corretto ma verifica la cosa sbagliata.

7c. auth-guard.spec.ts — testare una funzione con inject()

```
it('nega l\'accesso e reindirizza a /login se non loggato', () => {
  const result = TestBed.runInInjectionContext(
    () => canActivateNewTask({} as any, {} as any)
  );
  expect(result).toBeInstanceOf(UrlTree);
});
```

Il guard usa `inject()` al suo interno, quindi va eseguito dentro un injection context. `{}` `as any` per route/state: il guard non li legge mai, quindi il tipo esatto è irrilevante qui — compromesso accettabile solo quando sai che quei parametri non vengono usati. `toBeInstanceOf(UrlTree)` verifica il **tipo** del risultato, non un valore preciso: modo robusto di confermare "è stato generato un redirect" senza legarsi ai dettagli esatti dell'URL.

7d. task-form.spec.ts — component test e simulazione DOM

```
const input: HTMLInputElement = fixture.nativeElement.querySelector('#title');
input.value = 'ab';
input.dispatchEvent(new Event('input'));
input.dispatchEvent(new Event('blur')); // marca il campo "touched"
fixture.detectChanges();
await fixture.whenStable();
```

PERCHÉ SERVE DISPATCHEVENT

Non esiste un modo per "chiamare" `[formField]` direttamente da un test: è una direttiva che ascolta eventi DOM nativi, come farebbe un `v-model` di Vue sotto il cofano. Bisogna impostare `.value` come farebbe il browser e sparare manualmente l'evento — senza `dispatchEvent('input')` Angular non si accorgerebbe del cambio, perché osserva l'evento, non la proprietà. L'evento `'blur'` marca il campo "touched": senza, l'errore di validazione non comparirebbe mai nel DOM anche con un valore invalido.

In zoneless nulla si propaga "da solo" nei test: dopo ogni evento simulato serve sempre `detectChanges()` + `whenStable()` prima di poter leggere il DOM aggiornato.

```
const navigateSpy = vi.spyOn(router, 'navigateByUrl');
// ... simula submit ...
expect(store.tasks().some(t => t.title === 'Comprare il latte')).toBe(true);
expect(navigateSpy).toHaveBeenCalled();
```

Uno spy osserva le chiamate senza sostituire il comportamento — qui serve solo la prova che il componente abbia *tentato* di navigare, senza dover verificare una vera navigazione di pagina in ambiente di test.

7e. app.spec.ts — smoke test

```
it('si crea correttamente', () => {  
  const fixture = TestBed.createComponent(App);  
  expect(fixture.componentInstance).toBeTruthy();  
});
```

Il test più basilare possibile: se `createComponent()` incontra un errore nel costruttore, nella DI, o nel template, lancia un'eccezione prima ancora di arrivare a `expect()`. In un progetto reale, se si rompe la configurazione dei provider globali in `app.config.ts`, questo è il test che fallisce per primo e indica subito dove guardare.

7f. task-list.spec.ts — il deadlock che abbiamo risolto insieme

Questo è il caso di studio più concreto di debugging reale fatto durante il ripasso — il progetto era stato scritto in sandbox senza mai eseguire `npm test`, e questo bug lo confermava.

BUG 1 — DEADLOCK SU WHENSTABLE()

Il test originale chiamava `await fixture.whenStable()` *prima* di rispondere alla richiesta HTTP mockata di `httpResource()`. In modalità `zoneless`, l'app non diventa "stabile" finché una richiesta HTTP pending non viene evasa — quindi l'attesa non si sarebbe mai risolta (timeout dopo 5s). **Fix:** rispondere alla richiesta (`httpMock.expectOne(...).flush(...)`) *prima* di attendere la stabilità, non dopo.

BUG 2 — PROVIDER MANCANTE

`TaskItem` (renderizzato dentro `TaskList`) usa `routerLink`, che inietta `ActivatedRoute` — il test falliva con `NG0201: No provider found for ActivatedRoute`. **Fix:** aggiungere `provideRouter([])` ai provider del `TestBed`, sul modello degli altri file di test del progetto.

Lezione generale: un progetto scritto "a mano" senza mai eseguire la suite può sembrare corretto ma nascondere bug di ordine delle operazioni (async) e di provider mancanti — esattamente il tipo di errori che una vera esecuzione dei test scova immediatamente.

8. Concetti generali aggiuntivi

8a. Incapsulamento dei componenti (View Encapsulation)

Un componente Angular — come una SFC Vue — raggruppa in un'unica unità coesa template, logica e stile. Chi lo usa dall'esterno interagisce solo attraverso la sua "API pubblica" (`input()` / `output()`), senza conoscerne i dettagli interni. Questo è l'incapsulamento generale, comune a qualsiasi componente in qualsiasi framework.

C'è però un secondo significato, più specifico: **l'incapsulamento CSS**, che Angular chiama **View Encapsulation**. Per default, gli stili scritti in un componente non "escono" a influenzare altri componenti, e viceversa. Angular lo ottiene con `ViewEncapsulation.Emulated` (il default): aggiunge un attributo univoco generato automaticamente a ogni elemento renderizzato da quel componente (es. `_ngcontent-abc-12`) e riscrive i selettori CSS del componente per richiedere anche quell'attributo — emulando lo scoping senza il vero Shadow DOM del browser.

CONFRONTO DIRETTO CON VUE

È esattamente lo stesso meccanismo di `<style scoped>` nelle SFC — anche Vue fa attribute-based scoping (con `data-v-xxxxxx` invece di `_ngcontent-xxx`), stessa idea, stessa implementazione concettuale. Se conosci `<style scoped>`, conosci già `ViewEncapsulation.Emulated`.

Le tre modalità

Angular (<code>encapsulation: ViewEncapsulation.XXX</code>)	Equivalente Vue
<code>Emulated</code> (default) — scoping via attributi CSS generati	<code><style scoped></code>
<code>None</code> — nessun incapsulamento, stile globale	<code><style></code> senza <code>scoped</code>
<code>ShadowDom</code> — vero Shadow DOM nativo del browser, isolamento anche del DOM	nessun equivalente diretto pronto all'uso

La prova pratica nel progetto Task Board

Per vederlo funzionare, è stato aggiunto un piccolo `styles: [...]` inline nel decoratore `@Component` di ogni componente — solo un colore del testo, niente di elaborato. Il punto interessante: **quattro componenti diversi usano lo stesso selettore `h2`** (`TaskList`, `TaskDetail`, `TaskForm`, `Login`), ognuno con un colore diverso — e non c'è alcuna interferenza tra loro, proprio grazie alla View Encapsulation.

```
// app.ts
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styles: [`h1 { color: #1d4ed8; }`],
})

// task-list.ts
@Component({
  selector: 'app-task-list',
  imports: [TaskItem],
  templateUrl: './task-list.html',
  styles: [`h2 { color: #0f766e; }`],
})

// task-detail.ts → styles: [`h2 { color: #c2410c; }`]
// task-form.ts → styles: [`h2 { color: #be123c; }`]
// login.ts → styles: [`h2 { color: #15803d; }`]
// task-item.ts (niente h2 nel template, quindi lo span del titolo):
// styles: [`span { color: #6d28d9; }`]
```

Screenshot dell'app dopo la modifica — titolo blu (App), "Lista task" verde-teal (TaskList), titoli dei task viola (TaskItem), ognuno isolato nel proprio componente:

Task Board

[Lista task](#) [Nuovo task](#) [Login](#)

Lista task

Completati: 1 / 5

- ☐ [delectus aut autem](#) Elimina
- ☐ [quis ut nam facilis et officia qui](#) Elimina
- ☐ [fugiat veniam minus](#) Elimina
- ☒ [et porro tempora](#) Elimina
- ☐ [laboriosam mollitia et enim quasi adipisci quia provident illum](#) Elimina

Esperimento: e se passassimo tutti i componenti a ShadowDom?

Provato dal vivo nel progetto (poi ripristinato a `Emulated`): basta aggiungere `encapsulation: ViewEncapsulation.ShadowDom` al decoratore `@Component` di ciascun componente (più l'import di `ViewEncapsulation` da `@angular/core`).

COSA SI ROMPE DAVVERO, E PERCHÉ

Lo Shadow DOM reale **blocca** gli stili esterni al componente — non solo quelli emulati via attributo, un vero confine del browser. Il progetto ha un `src/styles.css` globale con regole come `.task { display: flex; gap: ...; border-bottom: ... }`, `.task.completed span { text-decoration: line-through }`, `nav a { margin-right: 1rem }`. Passando tutti i componenti a `ShadowDom`, nessuna di queste regole raggiunge più gli elementi dentro i rispettivi shadow root: i link di navigazione restano appiccicati senza spaziatura, le righe dei task perdono layout/bordi, la sottolineatura sui completati sparisce. **La logica applicativa (signal, httpResource, store) resta perfettamente intatta** — è solo il CSS globale a non "entrare" più, che è esattamente lo scopo dello Shadow DOM.

Task Board

[Lista task](#)[Nuovo task](#)[Login](#)

Lista task

Completati: 1 / 5

- ☐ [delectus aut autem](#) Elimina
- ☐ [quis ut nam facilis et officia qui](#) Elimina
- ☐ [fugiat veniam minus](#) Elimina
- ☒ [et porro tempora](#) Elimina
- ☐ [laboriosam mollitia et enim quasi adipisci quia provident illum](#) Elimina

E I TEST?

5 test su 13 falliscono (`app.spec.ts` , `task-form.spec.ts` ×3, `task-list.spec.ts`) con errori tipo `Cannot read properties of null: fixture.nativeElement.querySelector(...)` non attraversa i confini reali dello Shadow DOM (bisognerebbe passare da `nativeElement.shadowRoot.querySelector(...)`). Un effetto collaterale spesso sottovalutato: adottare `ShadowDom` in un progetto esistente richiede anche di riscrivere i test DOM-based, non solo lo stile.

Conclusione pratica: `ShadowDom` è utile quando l'isolamento deve essere reale e totale (es. un widget/Web Component distribuito su siti terzi di cui non controlli il CSS globale), ma richiede di NON affidarsi a fogli di stile globali — ogni componente deve portarsi dietro tutto ciò che gli serve. Per un'app "normale" come questa, `Emulated` (il default) resta la scelta giusta: isolamento sufficiente, nessuna delle due rotture viste sopra. Nel progetto è stato ripristinato `Emulated` dopo l'esperimento.

8b. Tailwind CSS installato per davvero: cosa succede in pratica

Continuando da §8a: **Tailwind CSS v4** (`tailwindcss` + `@tailwindcss/postcss`) è stato installato nel progetto per verificare dal vivo l'interazione con `Emulated` (la modalità in uso). Setup v4, niente `tailwind.config.js` obbligatorio:

```
npm install -D tailwindcss @tailwindcss/postcss

// .postcssrc.json (nuovo file, root del progetto)
{ "plugins": { "@tailwindcss/postcss": {} } }

/* src/styles.css, prima riga */
@import "tailwindcss";
```

Poi tutti i template sono stati riscritti con classi utility (`flex` , `rounded-lg` , `border` , `text-slate-500` , ecc.) al posto delle poche regole custom in `styles.css` . I colori dimostrativi per-componente di §8a (`styles: [...]` con un colore fisso per componente) sono stati **rimossi**, superati da un design coerente vero.

CONFERMA PRATICA DELLA TEORIA DI §8A

Esattamente come previsto: le classi utility Tailwind nei template funzionano senza alcun attrito sotto `Emulated` — nessuna configurazione aggiuntiva, nessun conflitto con l'incapsulamento.

Task Board

[Lista task](#) [Nuovo task](#) [Login](#)

Lista task

Completati: 1 / 5

☐ delectus aut autem	Elimina
☐ quis ut nam facilis et officia qui	Elimina
☐ fugiat veniam minus	Elimina
☒ et porro tempora	Elimina
☐ laboriosam mollitia et enim quasi adipisci quia provident illum	Elimina

Lista task dopo il redesign: card, righe con checkbox/bottone allineati, strikethrough sui completati.

Tre bug reali scoperti installando Tailwind (non ipotetici)

1. IL DEV SERVER NON VEDEVA LA NUOVA CONFIGURAZIONE POSTCSS

Dopo `npm install` e la creazione di `.postcssrc.json`, l'app continuava a renderizzarsi senza alcuno stile Tailwind — `styles.css` restava a poche centinaia di byte nei log di build. Causa: il processo `ng serve` era già in esecuzione da *prima* dell'installazione, e non rileva un plugin PostCSS aggiunto a runtime. **Fix:** riavviare completamente il dev server. Dopo il riavvio, `styles.css` è passato da ~0,6 KB a 16,6 KB nei log di build — segno che Tailwind stava generando CSS per davvero.

2. PAGINA BIANCA SUI DEEP-LINK DIRETTI (BUG PREESISTENTE, SCOPERTO PER CASO)

Navigando direttamente a `/tasks/1` (non passando dalla lista), la pagina risultava completamente bianca. Causa: `src/index.html` non aveva `<base href="/">` nell'`<head>` — il tag che il CLI di Angular genera sempre di default. Senza, gli URL relativi (`styles.css`, `main.js`) si risolvono rispetto al path corrente: su `/tasks/1`, `styles.css` diventa `/tasks/styles.css` (404 → fallback a `index.html` → errore di MIME type → foglio di stile e script rifiutati dal browser). Non c'entra nulla con Tailwind: è un bug preesistente del progetto scritto in sandbox, mai emerso prima perché non era mai stata provata la navigazione diretta a una rotta con parametro. **Fix:** aggiunto `<base href="/">` in `index.html`.

Task Board

[Lista task](#) [Nuovo task](#) [Login](#)

Login

Non sei loggato. La rotta "Nuovo task" è protetta da un guard funzionale.

Login (finto, un solo click)

Pagina di login dopo il redesign (raggiunta anche indirettamente: navigare a /new da disconnessi ci finisce tramite il guard).

3. ROUTERLINKACTIVE NON EVIDENZIAVA LA ROTTA ATTIVA (SCOPERTO DALL'UTENTE)

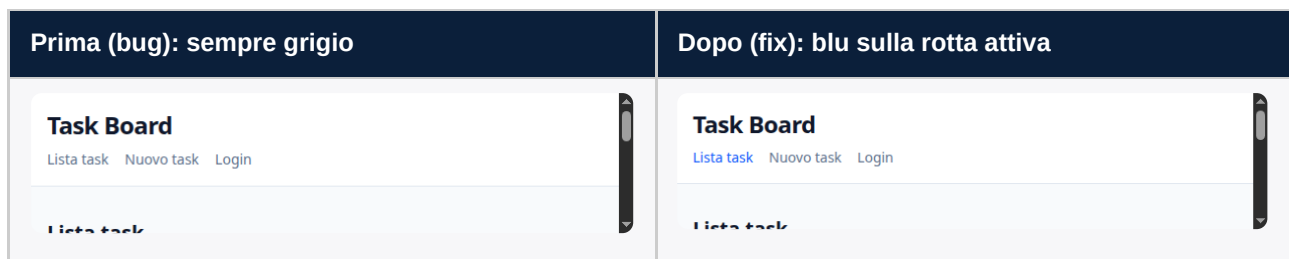
Il link della rotta corrente restava sempre grigio invece di colorarsi. Causa:

`routerLinkActive="text-blue-600"` **aggiunge** quella classe quando la rotta è attiva, ma non rimuove la classe statica `text-slate-500` già presente sull'elemento — le due finiscono entrambe nell'attributo `class` insieme. Con specificità CSS identica tra due utility Tailwind, a decidere è l'ordine nel foglio compilato, non l'ordine nel DOM: `.text-slate-500` compariva dopo `.text-blue-600` nell'output di Tailwind, quindi vinceva sempre lei, a prescindere da quale rotta fosse attiva.

```
<!-- prima: le due classi colore possono coesistere sullo stesso elemento -->
<a routerLink="/" routerLinkActive="text-blue-600"
  class="text-slate-500 ...">Lista task</a>

<!-- dopo: #rla espone .isActive, [class.x] rende le due classi
  colore reciprocamente esclusive — non coesistono mai -->
<a routerLink="/" routerLinkActive #rla="routerLinkActive"
  [class.text-blue-600]="rla.isActive"
  [class.text-slate-500]="!rla.isActive"
  class="...">Lista task</a>
```

Fix: usare la variabile template `#rla="routerLinkActive"` (che espone `.isActive`) con due `[class.xxx]` a esclusione reciproca, invece di affidarsi alla stringa passata a `routerLinkActive`. Lezione generale: quando una classe "attiva" condizionale tocca la stessa proprietà CSS di una classe statica di base, in Tailwind (o comunque con utility a specificità piatta) le due vanno rese mutuamente esclusive esplicitamente — non basta "aggiungere" la classe attiva sopra.



8c. Design token: cosa sono e come li usiamo

Un **design token** è un valore di design (colore, spaziatura, dimensione del font, raggio dei bordi, ombra...) astratto in una variabile con un **nome semantico**, invece di un valore letterale ripetuto ovunque nel codice. Invece di scrivere `color: #1a1a1a` in venti punti diversi, si definisce **una volta sola** `--color-text-strong: #1a1a1a` e la si riferenzia ovunque con `var(--color-text-strong)`. Il termine viene dal mondo dei design system (reso popolare da strumenti come il Lightning Design System di Salesforce, metà anni 2010): l'idea è che il token sia l'unità atomica di una decisione di design — un livello di indirizzione tra "cosa è stato deciso" e "dove viene applicato nel codice". Cambiare un token in un solo posto propaga la modifica ovunque venga usato.

I LIVELLI TIPICI IN UN DESIGN SYSTEM MATURO

- **Token globali/primitivi:** valori grezzi, es. `--blue-600: oklch(...)` — il "vocabolario" completo disponibile.
- **Token semantici:** nomi che descrivono l'USO, non il valore, es. `--color-accent: var(--blue-600)` — permette di cambiare "quale colore è l'accent" senza toccare i punti d'uso (utile per il dark mode: lo stesso token semantico punta a un primitivo diverso a seconda del tema).
- **Token di componente:** ancora più specifici, es. `--button-primary-bg: var(--color-accent)`.

Nel progetto Task Board abbiamo fatto una versione semplificata a un solo livello: i nostri `--ds-color-*` sono già "semantici" nel nome (`--ds-color-text-strong`, `--ds-color-accent`...) ma il valore è diretto, senza un ulteriore layer di alias verso primitivi grezzi. Scelta pragmatica adeguata alla scala di questo progetto — un design system più grande aggiungerebbe quel livello in più per la flessibilità del multi-tema.

Come sono usati nel progetto

Fonte unica in `src/styles.css`, dentro un blocco `:root { ... }` — il "livello globale" del sistema. Categorie definite: colori (`--ds-color-*`), raggio dei bordi (`--ds-radius-*`), tipografia (`--ds-text-*` / `--ds-font-*`), ombre (`--ds-shadow-sm`). Nota: la *spaziatura* non è stata tokenizzata sistematicamente — i componenti usano valori diretti in rem (`padding: 0.75rem 1rem`). Non tutto deve per forza diventare un token: per un progetto di queste dimensioni sarebbe stato over-engineering creare una scala di spaziatura completa per un pugno di valori usati due o tre volte ciascuno.

I valori sono stati "sniffati" dal CSS compilato di Tailwind (§8b) invece che inventati, per garanzia di parità visiva 1:1 con la versione precedente. Il consumo è distribuito: ogni componente li referencia con `var(--ds-color-accent)` dentro il proprio `styles: [...]`.

LA GERARCHIA A TRE LIVELLI DI QUESTO PROGETTO

1) **Token** (`styles.css`, globali) → i valori grezzi. 2) **Pattern condivisi** (`design-system/shared.styles.ts`, `.btn` / `.page-title` / ...) → combinazioni riutilizzabili di quei token. 3) **Applicazione** (classi nei singoli template) → dove i pattern vengono effettivamente usati. È la stessa idea concettuale di un file di variabili SCSS/design tokens + una libreria di componenti Vue che li consuma — solo che qui, per motivi che diventano chiari nella prossima sezione, il livello 2 deve essere fisicamente duplicato per componente invece che importato da un unico posto.

8d. Shadow DOM: come funziona, confronto con Vue, e relazione con i design token

Lo **Shadow DOM** è una feature nativa della piattaforma web (parte della spec dei Web Components, insieme a Custom Elements e HTML Templates), supportata nativamente da tutti i browser moderni — non è un'invenzione di Angular. Crea un albero DOM "nascosto" (uno `shadowRoot`) attaccato a un elemento host: il contenuto al suo interno è isolato dal resto del documento in entrambe le direzioni — gli stili globali del documento non entrano (con l'eccezione delle custom property, vedi sotto), e gli stili definiti dentro lo shadow root non escono. L'API grezza è `element.attachShadow({ mode: 'open' | 'closed' })`: `'open'` permette di accedere a `element.shadowRoot` dall'esterno (come abbiamo fatto nei test e negli script di verifica); `'closed'` lo nasconde completamente, usato raramente.

Come funziona specificamente in Angular

- Ogni componente con `encapsulation: ViewEncapsulation.ShadowDom` ottiene il proprio shadow root **indipendente** al momento della creazione dell'istanza — non è una proprietà del template, è per-istanza. Angular chiama `attachShadow({ mode: 'open' })` dietro le quinte (verificato empiricamente: `shadowRoot.mode === "open"` nel nostro script di verifica).
- Gli array `styles` / `styleUrls` del componente vengono iniettati come elementi `<style>` dentro **ciascuno** di quegli shadow root — con `styles: [sharedStyles, '...']` (due stringhe) otteniamo infatti 2 `<style>` per shadow root, confermato via CDP.
- **Annidamento**: un componente ShadowDom che ne renderizza un altro (es. `TaskList` che renderizza `TaskItem`) produce shadow root **annidati** — ognuno crea il proprio confine indipendentemente da dove si trova nell'albero. Verificato in questo progetto: `app-root` → `app-task-list` → `app-task-item`, tre shadow root separati, uno dentro l'altro.
- `document.querySelector` (e i suoi cugini) **non attraversano** il confine — bisogna scendere esplicitamente con `.shadowRoot.querySelector(...)` un livello alla volta, sia nei test (`fixture.nativeElement.shadowRoot!.querySelector(...)`) sia in qualunque script esterno.

CONFRONTO CON VUE

Vue **non** usa Shadow DOM per le sue Single File Component di default: `<style scoped>` è l'equivalente di `ViewEncapsulation.Emulated` di Angular — entrambi emulano l'isolamento via attributi CSS generati, senza vero Shadow DOM (vedi §8a). Vue *può* usare Shadow DOM vero, ma solo quando si costruisce esplicitamente un Web Component nativo con `defineCustomElement()` (Vue 3): a quel punto Vue avvolge il componente in un vero `customElements.define(...)` con `attachShadow()` dietro le quinte — stesso meccanismo di piattaforma usato da Angular.

La differenza concettuale non è tecnica ma di *default d'uso*: in Angular, `ShadowDom` è una delle tre opzioni ordinarie di styling per **qualsiasi** componente della tua app (`Emulated` / `None` / `ShadowDom` , un flag facilmente intercambiabile — puoi mescolare componenti `Emulated` e `ShadowDom` nella stessa app, come in questo stesso progetto durante la migrazione). In Vue, Shadow DOM è legato specificamente al percorso "sto costruendo un Web Component universale da distribuire anche fuori da Vue" — un caso d'uso diverso, non un'opzione ordinaria per i componenti interni della tua applicazione Vue.

La relazione tra custom property CSS, design token, e Shadow DOM

IL COLLEGAMENTO CHIAVE DI TUTTO QUESTO CAPITOLO

Le **CSS custom property attraversano i confini dello Shadow DOM**, per via delle normali regole di ereditarietà CSS — lo shadow boundary non le blocca, si comportano come `color` o `font-family` , che passano di padre in figlio anche attraverso quel confine. Le **classi CSS invece non lo attraversano mai**: il matching dei selettori è strutturale (basato sulla posizione dell'elemento nell'albero DOM), e lo shadow boundary tronca quell'albero agli occhi del CSS esterno. È il motivo esatto per cui Tailwind si è rotto nel primo esperimento (§8b/§7 del capitolo precedente) — le sue classi utility non potevano più raggiungere gli elementi dentro gli shadow root — mentre i nostri design token (custom property in `:root`) hanno continuato a funzionare senza problemi in **entrambi** gli esperimenti.

Conseguenza architetturale pratica, esattamente quella costruita in questo fork: **i design token vivono bene come variabili CSS globali** (attraversano i confini shadow gratuitamente, nessuno sforzo extra richiesto); **i pattern/classi che li consumano devono vivere localmente in ogni componente** (non attraversano mai i confini, vanno duplicati — da qui il pattern `shared.styles.ts` di §8c, DRY nel sorgente ma fisicamente ripetuto nel compilato). Due livelli, due comportamenti diversi rispetto allo Shadow DOM, e la separazione netta tra `styles.css` (solo token) e `shared.styles.ts` (solo classi) in questo progetto rispecchia esattamente questa distinzione.

8e. L'esperimento ShadowDom, capitolo 2: stavolta funziona

Nel capitolo precedente (§7, sul progetto originale con Tailwind) attivare `ShadowDom` su tutti i componenti aveva rotto sia il CSS globale sia 5 test su 13. Con la base costruita nei §8c/§8d di questo capitolo (design token globali + classi duplicate per componente, niente più dipendenza da un foglio di

stile globale con classi), lo stesso esperimento è stato ripetuto in `angular-vue-crash-project-full-encapsulation` — stavolta con esito completamente diverso.

VERIFICA STRUTTURALE VIA CDP (CHROME DEVTOOLS PROTOCOL)

Uno script Node con l'API `WebSocket` nativa (nessuna libreria come Puppeteer/Playwright installata su questa macchina) ha confermato shadow root **veri e annidati**:

```
{
  "appRootHasShadow": true,
  "appRootShadowMode": "open",
  "taskListFoundInsideAppShadow": true,
  "taskListHasOwnShadow": true,
  "taskItemFoundInsideTaskListShadow": true,
  "taskItemHasOwnShadow": true,
  "taskItemTitleInsideItsOwnShadow": "delectus aut autem",
  "styleTagsInAppShadow": 2,
  "styleTagsInTaskItemShadow": 2
}
```

Tre shadow root distinti (`app-root` → `app-task-list` → `app-task-item`), ciascuno con i propri 2 `<style>` (uno per `sharedStyles` , uno per il CSS specifico del componente) — esattamente la teoria di §8d confermata sul DOM reale in esecuzione.

Task Board

[Lista task](#) [Nuovo task](#) [Login](#)

Lista task

Completati: 1 / 5

☐ delectus aut autem	Elimina
☐ quis ut nam facilis et officia qui	Elimina
☐ fugiat veniam minus	Elimina
☒ et porro tempora	Elimina
☐ laboriosam mollitia et enim quasi adipisci quia provident illum	Elimina

Lista task con ShadowDom reale attivo su tutti i componenti — visivamente indistinguibile dalla versione Emulated.

Dettaglio task (dati reali, attraverso 3 shadow boundary)	Errore di validazione (dentro lo shadow root del form)
Task Board Lista task Nuovo task Login delectus aut autem Stato: da fare Segna come completato Elimina	Task Board Lista task Nuovo task Login Nuovo task Titolo ab Il titolo deve avere almeno 3 caratteri. Crea task

COSA SI È COMUNQUE DOVUTO SISTEMARE: I TEST

5 test su 13 sono falliti di nuovo — stesso sintomo del primo esperimento (`fixture.nativeElement.querySelector(...)` non trova nulla), ma stavolta per un motivo strutturale ovvio e non legato al CSS: quel metodo non attraversa mai lo shadow boundary reale, a prescindere da quanto sia ben fatto il CSS. Fix identico in tutti i casi: aggiungere `.shadowRoot!` prima di `.querySelector(...)` (`app.spec.ts` , `task-form.spec.ts` ×3, `task-list.spec.ts`). 13/13 verdi dopo il fix — l'unica rottura era di natura sintattica nei test, non un problema del codice applicativo.

Conclusione: la differenza tra i due esperimenti non è nello Shadow DOM in sé (si comporta identicamente in entrambi i casi, come da spec) — è nell'architettura CSS che lo circonda. Con Tailwind (foglio globale di classi) ShadowDom rompe tutto; con design token globali + classi duplicate per componente, ShadowDom funziona in modo trasparente, senza alcuna concessione visiva.

9. Appendice — note operative (setup ambiente)

Per far girare il progetto Task Board da zero, sono serviti alcuni aggiustamenti rispetto al `package.json` originale (scritto in sandbox senza accesso al registry npm):

- `typescript: ^5.6.0 → ^6.0.0` (richiesto da `@angular/build@22`)
- `vitest: ^3.0.0 → ^4.0.0` (richiesto da `@angular/build@22`)
- aggiunta `@types/node` (mancante, richiesta da `tsconfig.spec.json`)
- Node.js aggiornato via nvm da v24.13.1 a v24.18.1 (Angular 22 richiede $\geq 24.15.0$ o $\geq 22.22.3$)

Con questi fix: `npm install`, `npm start` (server su `http://localhost:4200/`) e `npm run test:ci` (13/13 test verdi) funzionano tutti correttamente.